

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

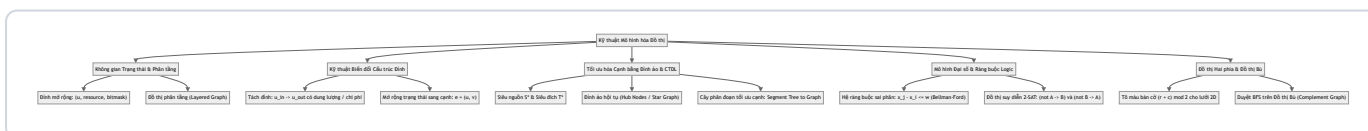
🎯 1. Tổng quan & Bản đồ Tư duy (Mindmap)

- Hầu hết các bài toán đồ thị học búa ở mức **Codeforces Div 2 D-F / Div 1 B-C, ICPC National & Regional** đều **KHÔNG** phát biểu dưới dạng: "*Cho đồ thị $G = (V, E)$, hãy chạy thuật toán X* ".

Định lý then chốt của Tư duy Đồ thị:

Đối tượng / Trạng thái $\iff$ Đỉnh (Vertices)

Quan hệ / Thao tác chuyển tiếp $\iff$ Canh (Edges)



Khi bài toán là một quá trình biến đổi từng bước:

- **Định:** Đại diện cho một **cấu hình hoàn chỉnh** của hệ thống tại một thời điểm.
- **Canh:** Đại diện cho một **nước đi / thao tác hợp lệ** chuyển từ cấu hình này sang cấu hình khác.

Ví dụ điển hình: Bài toán Đong nước 2 bình (Water Jug Puzzle)

Có 2 bình dung tích A và B . Ban đầu cả 2 bình rỗng. Các thao tác hợp lệ: Đổ đầy một bình, làm rỗng một bình, hoặc rót từ bình này sang bình kia đến khi đầy hoặc hết.

- Không gian trạng thái là cặp số nguyên (x, y) với $0 \leq x \leq A$ và $0 \leq y \leq B$.
- Tổng số đỉnh tối đa chỉ là $(A + 1)(B + 1)$.
- Mỗi trạng thái có tối đa 6 cạnh đi ra (tương ứng 6 thao tác).
- Tìm số thao tác ít nhất để đạt lượng nước $C \implies$ **Chạy BFS tìm đường đi ngắn nhất trên đồ thị trạng thái!**

2.2. Đồ thị Phân tầng (Layered Graph / Graph State Expansion)

Xét bài toán: "Tìm đường đi ngắn nhất từ 1 tới N . Trên hành trình, bạn có tối đa K lần sử dụng vé giảm giá (giảm trọng số cạnh về 0 hoặc một nửa)".

- Nếu chỉ lưu $\text{dist}[u]$, thuật toán Dijkstra sẽ thất bại vì trạng thái không thỏa mãn tính chất Markov (khoảng cách tối ưu tới u phụ thuộc vào việc đã dùng bao nhiêu vé trước đó).
- **Mô hình hóa phân tầng:**
 - Nhân bản đồ thị thành $K + 1$ tầng $(0, 1, \dots, K)$. Tầng k đại diện cho trạng thái: *Đang ở đỉnh u và đã sử dụng đúng k vé.*
 - Tổng số đỉnh: $N \cdot (K + 1)$.
 - **Cạnh nội tầng (không dùng vé):** Cạnh $(u, k) \xrightarrow{w} (v, k)$ với mọi $0 \leq k \leq K$.
 - **Cạnh liên tầng (dùng vé nhảy tầng):** Cạnh $(u, k) \xrightarrow{0} (v, k + 1)$ với mọi $0 \leq k < K$.
 - Chạy Dijkstra tiêu chuẩn trên đồ thị phân tầng này với điểm xuất phát $(S, 0)$ và đích là $\min_{0 \leq k \leq K} \text{dist}[(T, k)]$.

Tầng 2 (đã dùng 2 vé):	$(u, 2)$	-----w----->	$(v, 2)$
	$\wedge$		$\wedge$
	$\emptyset \mid$		$\emptyset \mid$
Tầng 1 (đã dùng 1 vé):	$(u, 1)$	-----w----->	$(v, 1)$
	$\wedge$		$\wedge$
	$\emptyset \mid$		$\emptyset \mid$
Tầng 0 (chưa dùng vé):	$(u, 0)$	-----w----->	$(v, 0)$

✂ 3. Kỹ thuật Biến đổi Cấu trúc Đỉnh (Vertex Transformations)

3.1. Kỹ thuật Tách Đỉnh (Vertex Splitting: $u_{\text{in}} \rightarrow u_{\text{out}}$)

Trong đồ thị chuẩn, trọng số và dung lượng thường gắn trên các **cạnh**. Nhưng nhiều bài toán thực tế lại đặt ràng buộc trên **đỉnh**:

- "Mỗi đỉnh u chỉ được phép đi qua tối đa C_u lần" (Vertex Capacity).
- "Mỗi khi ghé thăm đỉnh u , bạn phải trả lệ phí qua trạm là P_u " (Vertex Cost).

Cách mô hình hóa:

Với mỗi đỉnh $u \in V$, ta tách thành 2 đỉnh riêng biệt:

1. Đỉnh đón nhận dòng vào: u_{in} (hoặc u).
2. Đỉnh phát ra dòng đi: u_{out} (hoặc $u + N$).
3. Nối một cạnh có hướng từ u_{in} tới u_{out} :
 - Trọng số cạnh nội bộ này bằng chi phí qua đỉnh: $w(u_{\text{in}}, u_{\text{out}}) = P_u$.
 - Dung lượng cạnh nội bộ này bằng số lần tối đa được qua đỉnh: $\text{cap}(u_{\text{in}}, u_{\text{out}}) = C_u$.
4. Mọi cạnh ban đầu $u \rightarrow v$ có trọng số w sẽ được nối từ **cực ra** của u tới **cực vào** của v :

$$u_{\text{out}} \xrightarrow{w} v_{\text{in}}$$

Đồ thị gốc: $(u) \text{ ----- } w \text{ ----- } (v)$

Đồ thị tách đỉnh: $[u_{\text{in}}] \text{ --- } P_u \text{ ---} \rightarrow [u_{\text{out}}] \text{ --- } w \text{ ---} \rightarrow [v_{\text{in}}] \text{ --- } P_v \text{ ---} \rightarrow [v_{\text{out}}]$

3.2. Chuyển Đổi Không gian từ Đỉnh sang Cạnh (Edge as Vertex)

Xét bài toán: "Tìm đường đi ngắn nhất, nhưng cấm đi liên tiếp qua bộ ba đỉnh $a \rightarrow b \rightarrow c$ ".

- Nếu dùng đỉnh thông thường, khi đứng tại b , ta không biết được đỉnh trước đó là a hay một đỉnh khác.
- **Kỹ thuật biến cạnh thành đỉnh**:
 - Coi mỗi **cạnh có hướng** (u, v) là một đỉnh mới trong đồ thị trạng thái (tổng cộng M đỉnh).
 - Có cạnh nối từ đỉnh cạnh (u, v) tới đỉnh cạnh (v, w) nếu và chỉ nếu (u, v, w) không phải là bộ ba bị cấm!

🌟 4. Tối ưu hóa Cạnh bằng Đỉnh ảo & Cấu trúc Dữ liệu

4.1. Siêu Nguồn (Super Source) & Siêu Đích (Super Sink)

Khi bài toán có **nhiều đỉnh xuất phát** $S = \{s_1, \dots, s_k\}$ và **nhiều đỉnh đích** $T = \{t_1, \dots, t_m\}$:

- Tạo đỉnh **Siêu Nguồn** S^* : Nối cạnh có hướng $S^* \xrightarrow{0} s_i$ tới mọi đỉnh nguồn.
- Tạo đỉnh **Siêu Đích** T^* : Nối cạnh có hướng $t_j \xrightarrow{0} T^*$ từ mọi đỉnh đích.
- Bài toán tìm đường đi ngắn nhất hoặc luồng cực đại giữa hai tập hợp được đưa về bài toán chuẩn giữa đúng 2 đỉnh đơn lẻ: S^* và T^* .

4.2. Kỹ thuật Đỉnh ảo Hội tụ (Hub Nodes / Star Graph Optimization)

Xét bài toán: Một mạng lưới gồm M tuyến xe buýt. Tuyến xe buýt thứ i đi qua một tập hợp K_i trạm dừng. Hành khách có thể lên xe tại bất kỳ trạm nào và xuống tại bất kỳ trạm nào trên cùng tuyến.

- **Cách mô hình hóa ngây thơ**: Với mỗi tuyến xe buýt, nối cạnh hai chiều giữa mọi cặp trạm (u, v) trên tuyến đó:

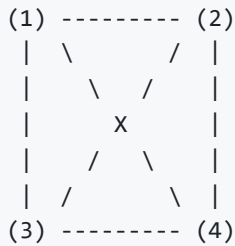
$$\text{Số cạnh cần tạo} = \sum_{i=1}^M \frac{K_i(K_i - 1)}{2} = O(M \cdot K^2)$$

Nếu $K_i = 10^5$, số cạnh lên tới $10^{10} \implies$ **Tràn bộ nhớ (MLE) và TLE tức thì!**

- **Giải pháp Đỉnh ảo Hội tụ (Hub Node):**

- Với mỗi tuyến xe buýt i , ta tạo ra đúng **1 đỉnh ảo trung tâm** Hub_i .
- Với mỗi trạm u trên tuyến i , ta chỉ nối:
- Cạnh lên xe: $u \xrightarrow{w_{lên}} Hub_i$.
- Cạnh xuống xe: $Hub_i \xrightarrow{w_{xuống}} u$.
- **Số cạnh mới tạo:** Chỉ là $2 \cdot K_i$ cạnh!
- Tổng số cạnh toàn đồ thị giảm ngoạn mục từ $O(K^2)$ xuống $O(K)$ **tuyến tính!**

Cách ngây thơ (Clique K^2 cạnh):



Dùng Đỉnh ảo Hub (2K cạnh):



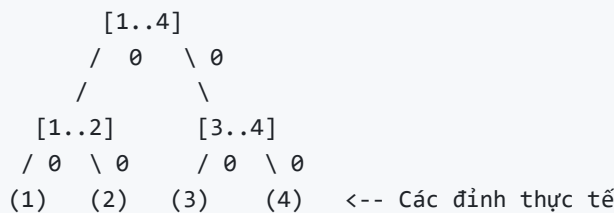
4.3. Kỹ thuật Segment Tree to Graph (Nối Đỉnh tới Đoạn)

Xét bài toán: Có N máy tính ($N \leq 10^5$). Có M quy tắc truyền tin dạng: Máy u có thể truyền tin đồng thời tới **t toàn bộ các máy trong đoạn** $[L, R]$ với chi phí W .

- Nối trực tiếp từ u tới từng đỉnh trong $[L, R]$ tốn $O(N \cdot M)$ cạnh $\implies$ TLE.
- **Kỹ thuật Cây phân đoạn Đồ thị (Segment Tree Graph):**
- Xây dựng một cây Segment Tree gồm $4N$ nút ảo đại diện cho các đoạn con.
- Trên cây, nối cạnh có hướng trọng số 0 từ **nút cha xuống 2 nút con**:

$$node \xrightarrow{0} node \cdot 2 \quad \text{và} \quad node \xrightarrow{0} node \cdot 2 + 1$$

- Các nút lá của cây tương ứng với các đỉnh thực từ 1 đến N .
- Khi cần nối cạnh từ đỉnh u tới đoạn $[L, R]$, đoạn $[L, R]$ được phân rã thành tối đa $O(\log N)$ nút trên Segment Tree. Ta chỉ cần nối từ u tới $O(\log N)$ nút này!
- **Số lượng cạnh:** Giảm từ $O(M \cdot N)$ xuống còn $O(4N + M \log N)$, cho phép chạy Dijkstra cực kỳ mượt mà!



Để nối $u \rightarrow [2..4]$: Nối $u \rightarrow (2)$ và $u \rightarrow [3..4]$ (Chỉ mất 2 cạnh thay vì 3!)

5. Mô hình Đại số & Ràng buộc Logic

5.1. Hệ Ràng buộc Sai phân (System of Difference Constraints)

Xét hệ gồm M bất phương trình tuyến tính với N ẩn số, mỗi bất phương trình có dạng:

$$x_j - x_i \leq w_k \quad (w_k \in \mathbb{R})$$

- Quan sát thấy dạng bất phương trình này **trùng khớp hoàn toàn** với điều kiện thư giãn tam giác trong bài toán đường đi ngắn nhất:

$$\text{dist}[j] \leq \text{dist}[i] + w(i, j) \iff \text{dist}[j] - \text{dist}[i] \leq w(i, j)$$

- **Quy tắc dựng đồ thị:**

- Mỗi ẩn số x_i là một đỉnh của đồ thị.
- Mỗi bất đẳng thức $x_j - x_i \leq w$ chuyển thành **cạnh có hướng từ i đến j với trọng số w :**

$$i \xrightarrow{w} j$$

3. Tạo siêu nguồn S^* , nối $S^* \xrightarrow{0} i$ tới mọi đỉnh.

4. Chạy thuật toán **Bellman-Ford / SPFA** từ S^* :

- Nếu đồ thị **có chu trình âm**: Hệ bất phương trình **vô nghiệm (Inconsistent)**!
- Nếu không có chu trình âm: Nghiệm hợp lệ chính là $x_i = \text{dist}[i]$.

5.2. Đồ thị Suy diễn trong Bài toán 2-SAT (Implication Graph)

Xét biểu thức logic gồm các mệnh đề tuyển 2 biến: $(x \vee y)$.

- Theo luật phản đảo trong logic học:

$$(x \vee y) \equiv (\neg x \implies y) \equiv (\neg y \implies x)$$

- **Quy tắc dựng đồ thị:**

- Với mỗi biến x , tạo 2 đỉnh: x và $\neg x$ (phủ định của x).
- Với mỗi mệnh đề $(u \vee v)$, thêm 2 cạnh suy diễn:

$$\neg u \longrightarrow v \quad \text{và} \quad \neg v \longrightarrow u$$

- Chạy thuật toán **Tarjan tìm Thành phần liên thông mạnh (SCC)**:

- Biểu thức thỏa mãn $\iff$ Không tồn tại biến x nào mà x và $\neg x$ nằm trong **cùng một SCC**!

6. Đồ thị Hai phía & Kỹ thuật Duyệt Đồ thị Bù

6.1. Tô màu Bàn cờ trên Lưới 2D (Grid Bipartite Graph)

Mọi lưới ô vuông $N \times M$ mà các bước di chuyển chỉ diễn ra giữa các ô kề cạnh (Manhattan distance lẻ) đều là một **Đồ thị Hai phía (Bipartite Graph)**:

- Tập đỉnh bên trái: Các ô có $(r + c)$ là số **chẵn** (ô Đen).
- Tập đỉnh bên phải: Các ô có $(r + c)$ là số **lẻ** (ô Trắng).
- Di chuyển 1 bước luôn chuyển từ ô Chẵn sang ô Lẻ hoặc ngược lại, **không bao giờ có cạnh nối giữa 2 ô cùng tính chẵn lẻ!**
- **Ứng dụng**: Bài toán xếp quân Mã trên bàn cờ, bài toán lát gạch Domino 1×2 , bài toán phủ ô vuông cực đại.

6.2. Duyệt BFS trên Đồ thị Bù (Complement Graph BFS)

Cho đồ thị $G = (V, E)$ với $V \leq 2 \cdot 10^5$, nhưng số cạnh $E \leq 2 \cdot 10^5$ (rất ít). Đồ thị bù $\overline{G}$ là đồ thị có cạnh giữa u và v khi và chỉ khi G **không có cạnh** giữa u và v .

- Số cạnh của $\overline{G}$ là $\frac{V(V-1)}{2} - E \approx 2 \cdot 10^{10} \implies$ Không thể xây dựng $\overline{G}$ trực tiếp!

- **Giải thuật BFS tối ưu với `std::set`**:

- Đưa toàn bộ các đỉnh chưa thăm vào một danh sách `std::set<int> unvisited = {1, 2, ..., n}`.
- Khi BFS tới đỉnh u , ta cần tìm các đỉnh $v \in unvisited$ mà $(u, v) \notin E$.
- Lần lượt duyệt qua các phần tử $v \in unvisited$:
 - Dùng cấu trúc `std::set` các đỉnh kề trong G để kiểm tra xem (u, v) có thuộc E không.
 - Nếu không thuộc E : Đỉnh v là đỉnh kề trong đồ thị bù $\implies$ Nạp v vào queue và **xóa ngay v khỏi `unvisited`**.
- Mỗi đỉnh bị xóa khỏi `unvisited` đúng 1 lần. Số lần kiểm tra thất bại tối đa bằng tổng số cạnh trong G ($2E$).
- Độ phức tạp: $O((V + E) \log V)$, duyệt toàn bộ đồ thị $2 \cdot 10^{10}$ cạnh chỉ trong 0.2 giây!

7. Bảng Tổng hợp Mô hình hóa Đồ thị

Bài toán gốc	Mô hình hóa Đồ thị	Kỹ thuật cốt lõi	Thuật toán áp dụng
Đổi trạng thái có điều kiện phụ	Đồ thị phân tầng (Layered Graph)	Nhân bản đỉnh (u, k)	Dijkstra / BFS
Ràng buộc lưu lượng / chi phí qua đỉnh	Tách đỉnh $(u_{in} \rightarrow u_{out})$	Chuyển đỉnh thành cạnh	Max Flow / Shortest Path
Nhiều nguồn, nhiều đích	Siêu nguồn S^* , Siêu đích T^*	Thêm 2 đỉnh ảo trọng số 0	BFS / Dijkstra / Dinic
Nhóm K đỉnh nối hoàn toàn với nhau	Đỉnh ảo hội tụ (Hub Node)	Giảm cạnh từ K^2 về $2K$	BFS / Dijkstra
Nối 1 đỉnh tới đoạn [L, R]	Cây phân đoạn đồ thị (SegTree Graph)	Phân rã đoạn thành $O(\log N)$ nút	Dijkstra / Topo DP

Bài toán gốc	Mô hình hóa Đồ thị	Kỹ thuật cốt lõi	Thuật toán áp dụng
Hệ bất đẳng thức $x_j - x_i \leq w$	Hệ ràng buộc sai phân	Cạnh có hướng $i \xrightarrow{w} j$	Bellman-Ford / SPFA
Mệnh đề logic 2 biến ($x \vee y$)	Đồ thị suy diễn 2-SAT	Cạnh $(\neg x \rightarrow y), (\neg y \rightarrow x)$	Tarjan SCC
Đồ thị dày bù của đồ thị thưa	Complement Graph	Dùng <code>std::set</code> xóa đỉnh đã thăm	BFS $O((V + E) \log V)$

⚠ 8. Cạm bẫy Phòng thi & Sai lầm Thường gặp (Pitfalls)

1. **Bùng nổ số lượng đỉnh / cạnh khi nhân bản trạng thái:**
2. Khi tạo đồ thị phân tầng $(u, resource)$, nếu $resource$ nhận quá nhiều giá trị (ví dụ: $K = 10^5$), tổng số đỉnh sẽ là $N \cdot K = 10^{10} \implies$ MLE ngay lập tức. Luôn kiểm tra xem K có nhỏ (≤ 20 hoặc ≤ 50) hay không trước khi nhân bản.
3. **Quên tạo cạnh hai chiều trong đồ thị vô hướng:**
4. Khi tách đỉnh hoặc thêm đỉnh ảo Hub cho đồ thị vô hướng, chiều đi và chiều về phải được nối đối xứng chính xác.
5. **Nhầm chiều cạnh trong Hệ ràng buộc sai phân:**
6. Bất đẳng thức là $x_j - x_i \leq w \iff x_j \leq x_i + w$. Cạnh bắt buộc phải đi từ i **sang** j ($i \rightarrow j$), KHÔNG PHẢI $j \rightarrow i$!
7. **Không xử lý được đồ thị bù nếu dùng ma trận kề:**
8. Quên rằng $V = 2 \cdot 10^5$ không thể khai báo `adj[V][V]`. Bắt buộc phải dùng kỹ thuật xóa đỉnh trên `std::set` hoặc danh sách liên kết.

📚 9. Tài liệu Tham khảo & Đọc thêm

1. **USACO Guide (Platinum / Gold Section):**
2. [Graph Modeling & State Space Search](#).
3. **Introduction to Algorithms (CLRS 4th Edition):**
4. *Section 22.4: Systems of difference constraints and shortest paths.*
5. **Competitive Programming 4 (Book 2)** — Steven Halim:
6. *Chapter 4: Advanced Graph Algorithms (State-Space Graphs, Vertex Splitting).*
7. **Codeforces Catalog:**
8. [Segment Tree to Graph Optimization Tutorials](#).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.